

Task 6: Subqueries and Nested Queries

ELEVATE LABS

SQL Developer Internship Deliverable

SUBMITTED BY

G.Pavana Deepika

DOMAIN

SQL Development

TOOLS USED

DB Browser for SQLite / MySQL

1. TASK OVERVIEW

Objectives:

- Implement scalar & correlated subqueries
- Utilize subqueries in SELECT, WHERE, & FROM
- Master filtering with IN, EXISTS, and comparison operators

Key Concepts Covered:

- Subqueries & Nested Filtering
- Derived Tables
- Correlated Subquery Logic
- Performance Implications

2. PRACTICAL SQL IMPLEMENTATIONS

Scenario A: Scalar Subquery in WHERE Clause

```
-- Find employees who earn more than the overall average salary
SELECT employee_id, first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Scenario B: Subquery in WHERE Clause (Using IN)

```
-- Find customers who have placed at least one order
SELECT customer_id, customer_name
FROM customers
WHERE customer_id IN (SELECT DISTINCT customer_id FROM orders);
```

Scenario C: Correlated Subquery (Using EXISTS)

```
-- Find departments that currently have assigned employees
SELECT department_id, department_name
FROM departments d
WHERE EXISTS ( SELECT 1 FROM employees e
WHERE e.department_id = d.department_id );
```

Scenario D: Subquery in SELECT Clause

```
-- Display employee salary alongside average company salary
SELECT first_name, salary,
(SELECT AVG(salary) FROM employees) AS avg_company_salary
FROM employees;
```

Scenario E: Derived Table (Subquery in FROM Clause)

```
-- Calculate average spend per customer using a derived table
SELECT AVG(total_spent) AS average_customer_spending
FROM ( SELECT customer_id, SUM(order_amount) AS total_spent
FROM orders
GROUP BY customer_id ) AS customer_totals;
```

3. INTERVIEW QUESTIONS & DETAILED SOLUTIONS

1. What is a subquery?

A subquery is an inner SQL query nested inside a larger outer query (such as SELECT, INSERT, UPDATE, or DELETE). It retrieves data that is used by the main query as a filtering condition, dynamic value, or source table.

2. What is the difference between a subquery and a join?

A **JOIN** combines columns from multiple tables side-by-side based on relationships and is generally optimized for performance across large datasets. A **Subquery** isolates or calculates specific data before returning it to the outer statement, making it useful for multi-step filtering or aggregation.

3. What is a correlated subquery?

A correlated subquery is an inner query that references columns from the outer query. Because of this dependency, the subquery must be evaluated repeatedly for each row evaluated by the outer query.

4. Can subqueries return multiple rows?

Yes, subqueries can return multiple rows when evaluated using set membership operators like IN, NOT IN, ANY, ALL, or presence operators like EXISTS.

5. How does EXISTS work?

EXISTS evaluates to TRUE as soon as the inner query returns at least one row. It performs short-circuit evaluation, stopping search operations early and making it highly efficient for existence checks.

6. How is performance affected by subqueries?

Non-correlated subqueries execute only once and carry minimal overhead. Correlated subqueries execute once per outer row, which can lead to $O(N^2)$ complexity and severe performance bottlenecks on large tables compared to JOINS.

7. What is a scalar subquery?

A scalar subquery is an inner query that returns exactly one row and one column (a single atomic value). It can be used anywhere single values or constants are allowed in SQL.

8. Where can we use subqueries?

Subqueries can be used in SELECT, FROM, WHERE, HAVING, JOIN clauses, as well as in data modification statements like INSERT, UPDATE, and DELETE.

9. Can a subquery be in the FROM clause?

Yes, subqueries in the FROM clause act as temporary tables (derived tables) that can be queried, aggregated, or joined just like physical database tables.

10. What is a derived table?

A derived table is an expression in the FROM clause that creates a virtual table during execution. Most SQL RDBMS require derived tables to be assigned an explicit alias name.